

Sentiment Analysis with Spark NLP vs Hugging Face in PySpark

1. Spark NLP Approach

Integration: It is developed natively on top of Apache Spark using native Spark ML Pipelines. Workflow:

1. Raw text is structured using DocumentAssembler.
2. UniversalSentenceEncoder for dense embeddings with pretraining.
3. A classification model called SentimentDLModel was developed using data from Twitter.

Benefits include full distribution, little bridging overhead, and spark optimisation. Limitations: There are fewer models and fewer fine-tuning choices.

2. Hugging Face Transformers in PySpark

Integration: Makes use of distributed inference or Pandas UDFs to use Hugging Face models in Spark.

1. The first step in the workflow is to load a pretrained transformer (such as distilbert-base-uncased-finetuned-sst-2-english).
2. Wrap inference in UDFs for Pandas.
3. Create a Spark DataFrame from the results.

Benefits include modular architectures, support for fine-tuning, and access to hundreds of models. Limitations: Requires more memory and engineering work.

3. Choosing Between Them

- Use Spark NLP for fast deployment, pretrained models, and seamless scaling in Spark.
- Use Hugging Face when model variety, cutting-edge architectures, or custom fine-tuning are priorities.

Feature	Spark NLP	Hugging Face in Spark
Model Variety	Limited to Spark NLP repo	Thousands of models available
Scaling	Native distributed execution	Needs Pandas UDF/batching
Fine-tuning Support	Minimal	Extensive (PyTorch/TF)
Ease of Setup	Simple in Spark pipeline	More engineering required